Aula — Laboratório

Implementação de um neurônio artificial e comparação de funções de ativação

Objetivos desta sessão

- Implementar em NumPy as funções de ativação (degrau, identidade, sigmoide, tanh, ReLU e Leaky ReLU) e um **neurônio** capaz de processar um vetor ou um lote de entradas.
- Comparar as ativações sobre as mesmas entradas: faixa de valores, centralização em zero, derivada e fração saturada.
- Reproduzir **portas lógicas** com pesos escolhidos à mão e mostrar, por busca em grade, que o XOR não tem solução com um único neurônio.
- Resolver o XOR com **duas camadas** e verificar numericamente que, sem ativação não linear, duas camadas colapsam em uma.
- Medir a **saturação** e o gradiente que desaparece (ou explode) ao atravessar várias camadas, e observar a ReLU morta.
- Conferir que um neurônio sigmoide é a **regressão logística** e um neurônio identidade é a regressão linear; implementar a softmax de forma estável.

1. Onde estamos

Na aula de conceitos vimos o modelo matemático do neurônio, o papel do viés, o limite do XOR e o catálogo das funções de ativação. Hoje o neurônio sai do papel: você vai escrevê-lo, compará-lo e quebrá-lo de propósito para enxergar em números o que a teoria afirma. Nenhum neurônio é **treinado** nesta sessão: os pesos são dados, escolhidos à mão ou copiados de um modelo ajustado. O treinamento do perceptron é o assunto do laboratório seguinte.

2. Material do laboratório

Arquivo	Para que serve
`lab_neuronio_demo.py` / `lab_neuronio_ativacoes.ipynb`	demonstração guiada, feita em conjunto no início da sessão (gera as três figuras desta apostila)
`lab_neuronio_exercicios.py`	arquivo do aluno: complete os `# TODO` e responda o RELATÓRIO
numpy, pandas, matplotlib, scikit-learn	bibliotecas usadas; nenhum arquivo de dados é necessário (a Iris vem com o scikit-learn)

O scikit-learn só aparece no Exercício 6, para ajustar uma regressão logística e uma linear de referência. Todo o restante é NumPy puro: **não use TensorFlow, PyTorch nem MLPClassifier**.

3. Roteiro da sessão

Etapa	O que fazer
1. Demonstração	executar o arquivo de demonstração e discutir cada saída: o neurônio conferido com o exemplo feito à mão, os mapas de saída, a busca em grade, o gradiente por camada, o XNOR em duas camadas e a softmax
2. Exercícios 1 e 2	funções de ativação, classe `Neuronio`, tabela comparativa e derivadas conferidas contra a derivada numérica
3. Exercícios 3 e 4	portas lógicas, busca em grade, escala dos pesos; XOR em duas camadas e colapso sem não linearidade
4. Exercício 5	gradiente por L camadas e ReLU morta
5. Exercícios 6 e 7 e relatório	neurônio na Iris (logística e linear), softmax estável e interpretação por escrito

4. O que a demonstração mostra

4.1. O neurônio em poucas linhas

A classe tem dois métodos. O operador `@` faz o produto escalar para um vetor e, sem mudar uma linha, o produto matriz-vetor para um lote inteiro:

```
class Neuronio:
    def __init__(self, w, b, ativacao="sigmoide"):
        self.w = np.asarray(w, dtype=float)
        self.b = float(b)
        self.ativacao = ativacao

    def z(self, X):  # soma ponderada + viés; X: (d,) ou (n, d)
        return np.asarray(X, dtype=float) @ self.w + self.b

    def saida(self, X):  # ativação aplicada a z
        return ATIVACOES[self.ativacao](self.z(X))
```

Com o exemplo feito à mão da aula de conceitos, x = (2; −1; 3), w = (0,5; 1,0; −0,2) e b = 0,3, o código devolve z = −0,300 e as saídas 0 (degrau), 0,426 (sigmoide), −0,291 (tanh), 0 (ReLU) e −0,003 (Leaky ReLU): exatamente os valores calculados no papel.

4.2. A mesma fronteira, saídas diferentes

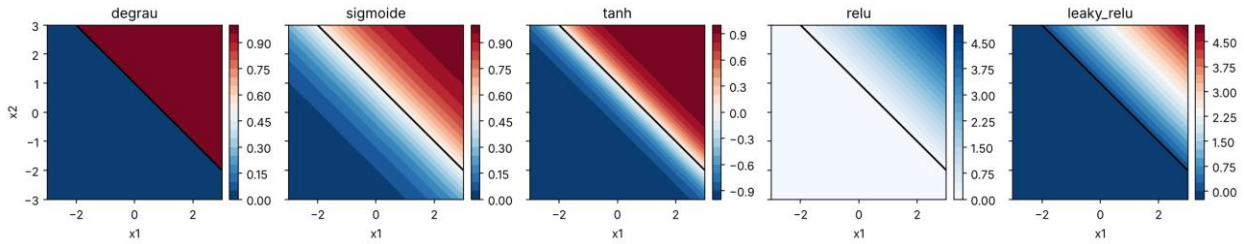


Figura 1 — saída do mesmo neurônio (w = (1; 1), b = −1) com cinco ativações. A linha preta é a fronteira z = 0.

A fronteira `z = 0` é idêntica nos cinco mapas. O que a ativação muda é a **forma** da saída ao redor dela: um salto (degrau), uma transição suave limitada (sigmoide e tanh) ou uma rampa ilimitada (ReLU e Leaky ReLU). No ponto B = (0,8; 0,6), com z = 0,4, as saídas são 1; 0,599; 0,380; 0,4 e 0,4.

4.3. Um neurônio não resolve o XOR; dois resolvem o XNOR

Na busca em grade com 343 combinações de (w_1, w_2, b), quatro reproduzem a AND e nenhuma reproduz o XOR. Com um neurônio oculto AND, um NOR e um de saída OR (o XNOR vale 1 quando as entradas são iguais), cada neurônio continua linear, mas a camada oculta leva os pontos a um espaço em que **uma reta separa as classes**:

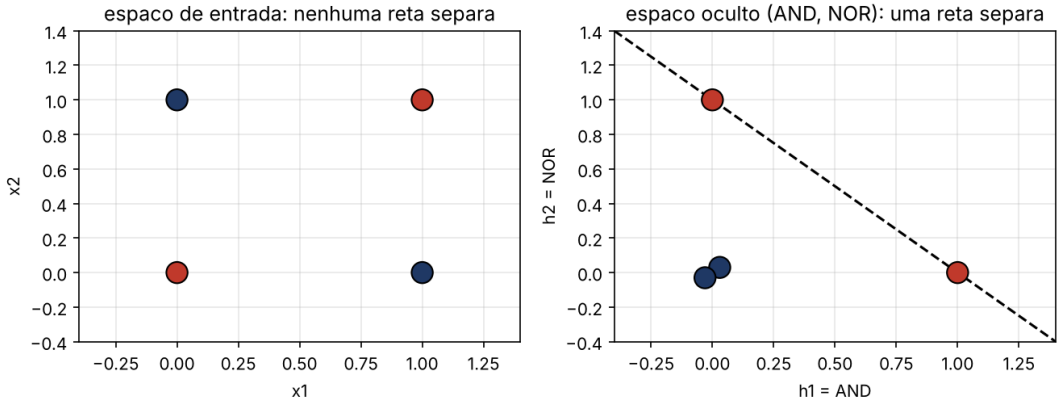


Figura 2 — à esquerda, o espaço de entrada (nenhuma reta separa); à direita, o espaço oculto (uma reta separa). As duas entradas de classe 0 caem no mesmo ponto.

Se trocarmos as ativações por identidade, W' = W_2W_1 = (0; 0) e b' = −1,5: a rede inteira devolve −1,5 para qualquer entrada. Sem não linearidade, camadas não acrescentam poder.

4.4. O gradiente atravessando a rede

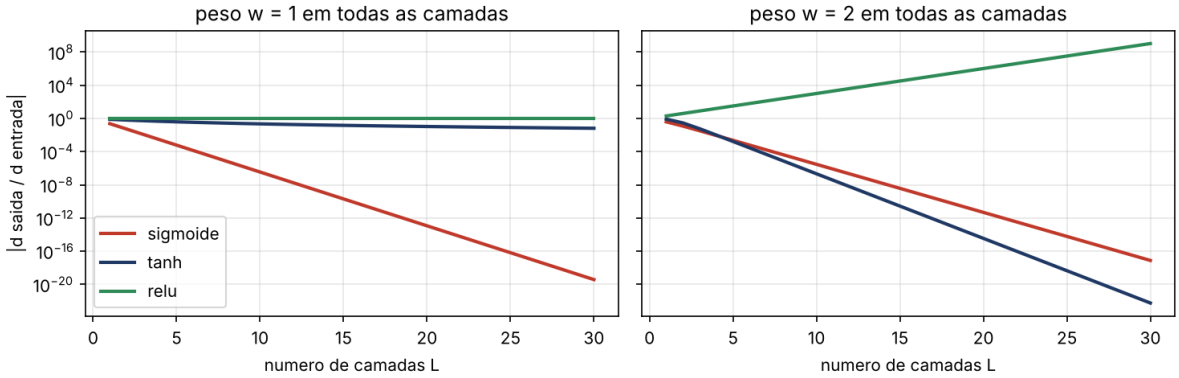


Figura 3 — |d saída / d entrada| em uma cadeia de L neurônios (escala logarítmica), com peso w em todas as camadas.

Com w = 2, em L = 10 camadas o gradiente da sigmoide é 3,0 × 10^{-6} e o da tanh, 2,1 × 10^{-7} (desaparece); o da ReLU é 1,0 × 10^{3} (**explode**). A ReLU elimina a saturação, mas não escolhe os pesos por você: a escala dos pesos importa tanto quanto a ativação.